

SymPy Bug Report

1 Bug 60: LerchPhi Left Limit Returns Divergent Endpoint

1.1 Minimal Reproducer

```
from sympy import lerchphi, limit, symbols

x = symbols("x")
print(limit(lerchphi(x, 1, 1), x, 1, dir="-"))
```

1.2 Observed Behavior

```
limit = lerchphi(1, 1, 1)
```

On the checked SymPy development version, the left-hand limit is returned as the endpoint expression `lerchphi(1, 1, 1)`. That endpoint is not a finite value of the original analytic expression; it is the divergent defining series after substituting $x = 1$.

1.3 Expected Behavior

The correct result is $+\infty$:

$$\lim_{x \rightarrow 1^-} \text{lerchphi}(x, 1, 1) = \infty.$$

For the representative case, applying SymPy's functional expansion first gives

$$\text{lerchphi}(x, 1, 1) = \frac{-\log(1-x)}{x},$$

whose left-hand limit at $x = 1$ is $+\infty$.

1.4 Mathematical Explanation

For $0 < x < 1$, the Lerch transcendent has the convergent series representation

$$\text{lerchphi}(x, 1, a) = \sum_{n=0}^{\infty} \frac{x^n}{n+a}.$$

In the representative case $a = 1$,

$$\text{lerchphi}(x, 1, 1) = \sum_{n=0}^{\infty} \frac{x^n}{n+1} = \frac{-\log(1-x)}{x}.$$

As $x \rightarrow 1^-$, the numerator $-\log(1-x)$ diverges to $+\infty$ and the denominator tends to 1. Therefore the left-hand limit is $+\infty$, not an unevaluated endpoint.

The returned expression `lerchphi(1, 1, 1)` is not an equivalent representation of the limit. At $x = 1$ the defining series becomes

$$\sum_{n=0}^{\infty} \frac{1}{n+1},$$

the harmonic series, which diverges. Returning the endpoint expression masks that divergence instead of reporting the limit.

1.5 Source-Level Diagnosis

The source-level diagnosis locates the immediate wrong step in `sympy/series/gruntz.py`, specifically in `mrvt_leadterm` around lines 520–538. The call path described in the artifact is: `limit(...)` constructs a `Limit` object and enters `Limit.doit` in `sympy/series/limits.py`; after the ordinary leading-term path cannot expand the shifted expression, `Limit.doit` calls `Gruntz`. For the left-hand finite limit, `Gruntz` substitutes $x = 1 - 1/z$ and analyzes

$$\text{lerchphi}\left(1 - \frac{1}{z}, 1, 1\right)$$

as $z \rightarrow \infty$.

The diagnosis identifies the following traced behavior:

```
e0 = lerchphi(1 - 1/z, 1, 1)
e0.rewrite("tractable", deep=True, limitvar=z) -> e0
mrvt_leadterm(e0, z) -> (lerchphi(1, 1, 1), 0)
limitinf(e0, z) -> lerchphi(1, 1, 1)
```

Thus the generic `Gruntz` leading-term machinery treats the special function as having exponent 0 and coefficient `lerchphi(1, 1, 1)`. The subsequent `limitinf` logic returns that coefficient unchanged, so the endpoint is substituted at a divergence point.

The special-function implementation contributes to the miss because `sympy/functions/special/zeta_functions.py` contains expansion rules that can expose this case: `lerchphi(x, 1, 1)` expands to $-\log(1-x)/x$. However, the diagnosis says `lerchphi` does not provide an `eval`, leading-term, or tractable rewrite hook that this limit path applies automatically near $x = 1$. A plausible fix direction identified by the artifact is to add `limit`, leading-term, or tractable rewrite support for `lerchphi` near $z = 1$, at least for the $a = 1$ specialization and the $s = 1$ logarithmic case, so the generic `Gruntz` path does not treat the divergent endpoint as a finite coefficient.

1.6 Additional Failing Instances

The independent verification checks the same left-hand limit for $a = 1, 2, 3, 4, 5, 6$. In each case, SymPy returns the endpoint expression `lerchphi(1, 1, a)`, while the expected result is $+\infty$.

Input / Expression	SymPy behavior	Expected behavior
<code>limit(lerchphi(x, 1, 1), x, 1, -)</code>	<code>lerchphi(1, 1, 1)</code>	$+\infty$
<code>limit(lerchphi(x, 1, 2), x, 1, -)</code>	<code>lerchphi(1, 1, 2)</code>	$+\infty$
<code>limit(lerchphi(x, 1, 3), x, 1, -)</code>	<code>lerchphi(1, 1, 3)</code>	$+\infty$
<code>limit(lerchphi(x, 1, 4), x, 1, -)</code>	<code>lerchphi(1, 1, 4)</code>	$+\infty$
<code>limit(lerchphi(x, 1, 5), x, 1, -)</code>	<code>lerchphi(1, 1, 5)</code>	$+\infty$
<code>limit(lerchphi(x, 1, 6), x, 1, -)</code>	<code>lerchphi(1, 1, 6)</code>	$+\infty$

The expected behavior is the same across this family. For every listed positive integer a , the terms are positive on $0 < x < 1$, and as $x \rightarrow 1^-$ the series tends to the divergent shifted harmonic series

$$\sum_{n=0}^{\infty} \frac{1}{n+a}.$$

The verification script also records large finite partial sums near $x = 1$, consistent with monotone growth toward $+\infty$, not convergence to a finite endpoint value.

1.7 Related Issues Assessment

The deduplication report classifies this as a new instance of a known failure mode. The closest public matches involve LerchPhi, summation endpoint behavior, and convergence conditions: SymPy issue 8412 reports a summation producing `lerchphi(1, 2, a)` followed by an endpoint substitution that gives `nan`, and issue 14219 includes finite sums containing `lerchphi(x, 1, k + 2)` with convergence restrictions around $|x| \leq 1$ and $x \neq 1$. Those reports are related to the broader LerchPhi endpoint/convergence family, but the artifact's best-effort deduplication check did not identify a public report for the direct left-hand limit `limit(lerchphi(x, 1, 1), x, 1, dir = -)` returning the unevaluated divergent endpoint. This exact reproducible case remains individually actionable as a SymPy issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import S, lerchphi, limit, symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_left_limit_lerchphi_s_one_at_one_is_positive_infinity(a):
    x = symbols("x")
    assert limit(lerchphi(x, 1, a), x, 1, dir="-") == S.Infinity
```